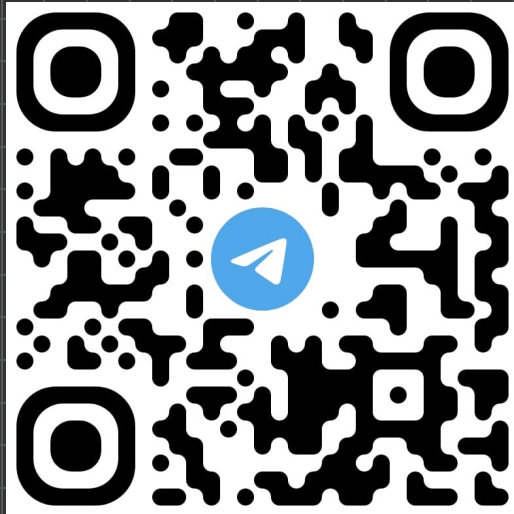


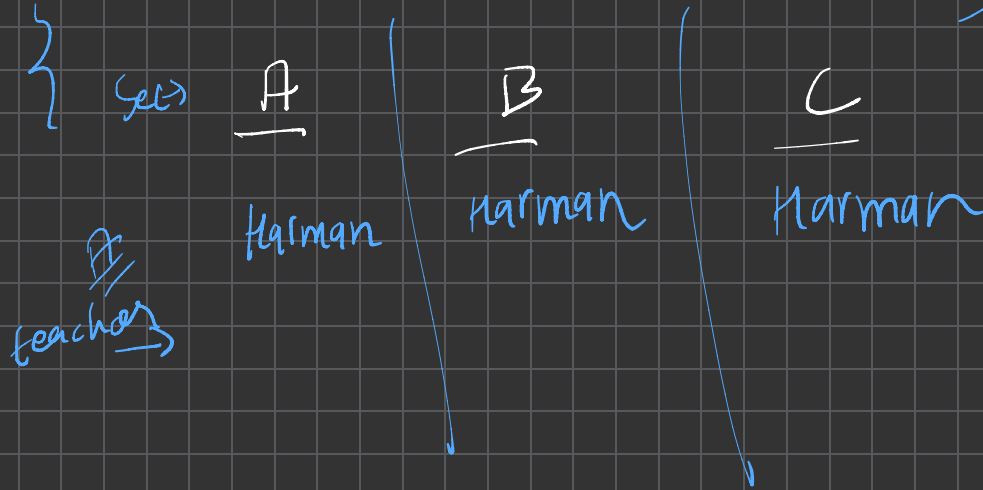
Telegram



Discord



scope → variable
↳ access



}

scopes variable

function

- Global scope
- function scope
- Block scope

internals of JS

global scope

↳ variable



globally access

→ BLOCK scope (let & const)

var X

→ { → why?

→ }

{

}

function scope → var

~~✓ var~~

function
scope

let ✓

block
scope

{
 } let

}

const ✓

block
scope

Block $\rightarrow$ function
func, for if {
}

call
involve

```
function counter(){
```

```
  let count = 0 ✓
```

```
  count = count + 1
```

```
  console.log(count);
```

```
}
```

```
counter()
```

```
counter()
```

independent

↑ 2
6
count

globally

↑ 2

```
let count = 0
```

```
function counter() {
```

```
  count = count + 1
```

```
  console.log(count);
```

9, 2

```
counter()
```

```
counter()
```

```
let count = 0

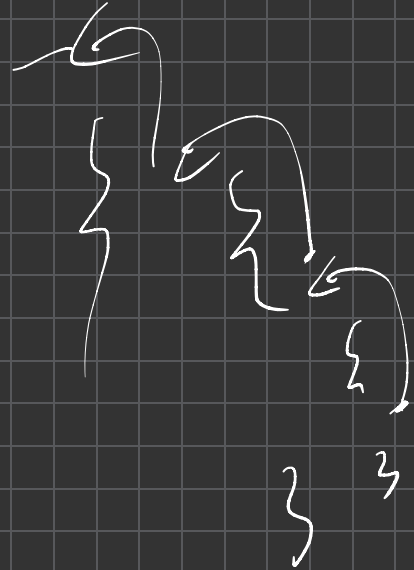
function counter() {
  count = count + 2
  console.log(count);
}

counter()
counter()
```

```
let count = 0  
  
function counter(){  
  count = count + 2  
  console.log(count);  
}
```

```
counter()  
counter()
```

$\boxed{2}$
count



✓

```
let count = 2
```

```
function counter() {  
  let count = 0  
  count = count + 2  
  console.log(count);  
}
```

```
counter()  
counter()
```

2
count
→ global

count
→ function()

programming

✓
var
→ let
→ const

→ syntax

Holisting

() variables & functions declaration
Top per chole gate hai

() let, var, const
() hoisted

and you can
use before
declaration & initialization

$\log(a)$ $\rightarrow$ undefined

$\text{var } a = 5; \text{ func}$

undefined $\leftarrow a()$

$\text{var } \overset{a}{a} = \text{func}() \{ \}$

How is code executed?

creation phase
(Memory allocation phase)

- variable
 - ↳ memory
- function
 - copy & paste into memory

Execution phase

- value assign
- $\text{clg}()$, +, -
- invoke function

var
↳ declare
↳ initialize : undefined

let & const
↳ declare
↳ X

creation	Execution
key : value	
<u>a : undefined</u>	
<u>b : <value unavailable></u>	

}
name :
email :
}

Temporal Dead Zone

var a = 9

let b = 10

variable creation

se lekr

↓ time

initialization tak

Memory allocation

a : undefined

b : <value unavailable>

const, let b = 10;

creation b : _____

Temporal Dead zone

```
let x = 6;  
function random(){  
  console.log(x);  
  let x = 3;  
}  
random();
```

Execution

b : 10

Temporal = Time se related

[6]

+ global

var

~~let~~ x = 6;

```
function random() {  
  console.log(x);  
  let x = 3  
}
```

var

random()

undefined

→ e

random()

memory
allocation

→ `let city = "Delhi"`

→ `function printCity(){
 console.log(city);
}`

→ Delhi
→ variable

fn = printCity

→ `function random(fn){
 → let city = "Varanasi"
 fn() →
}`

→ `random(printCity)`

→ callback function

lexical scoping

lexical environment

```
function random(fn) {  
  let city = "Varanasi"  
  function printCity() {  
    console.log(city);  
  }  
  return printCity  
}
```

```
console.log(random())
```

[function: print]

```
function outter(){
```

```
  { function inner(){  
    return 10;  
  }  
}
```

```
  inner() // right
```

```
  return inner
```

```
  return inner()
```

```
}
```

copy & return
function definition

function call
↓
undefined

return

10

lexical environment

inner function

↳ variable
+ parent ke
environment
→ outter()

